



NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

STR

TP3 – LAB#1

Petri Nets

2022 - 2023

PART 1:

Software installation and development of first PN models

- ☐ Revisiting Petri net Modeling
- ☐ Installing HPSim software
- ☐ Implementation of examples from theoretical classes
- ☐ Development of several modules to training on PN concepts

PART 2:

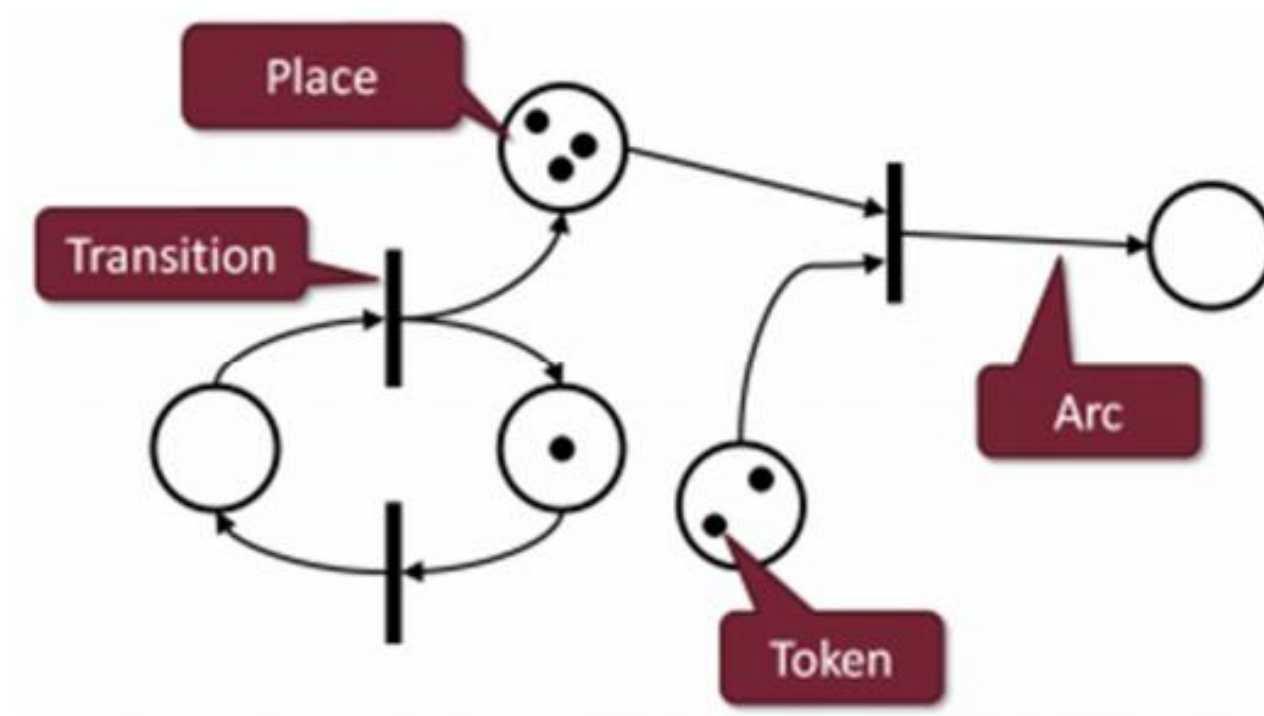
DLL integration in HPSim software

- ☐ Installing new version of HPSim software
- ☐ Creating a DLL
- ☐ Using the DLL in HPSim
- ☐ First examples

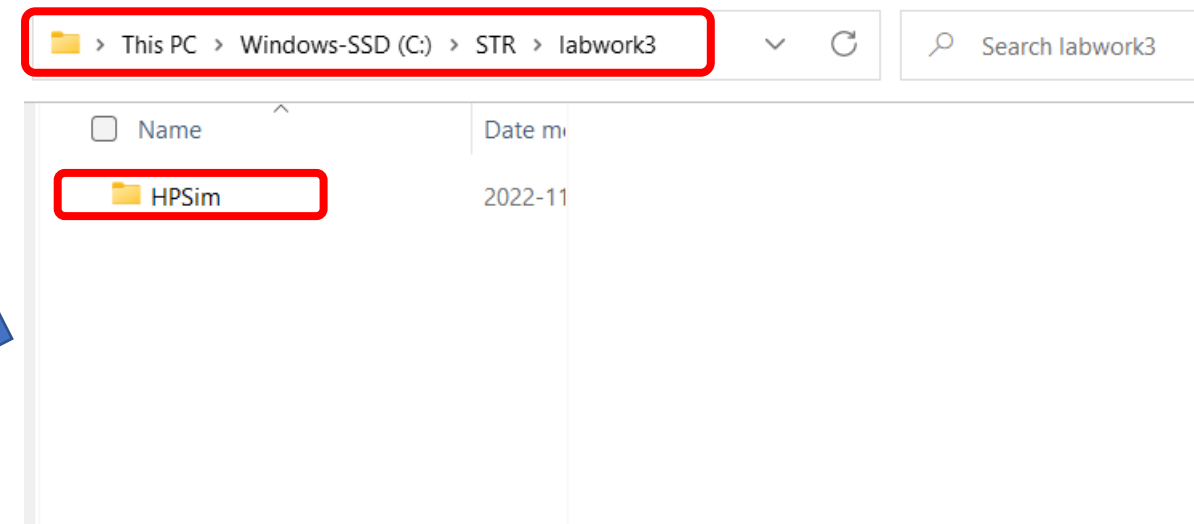
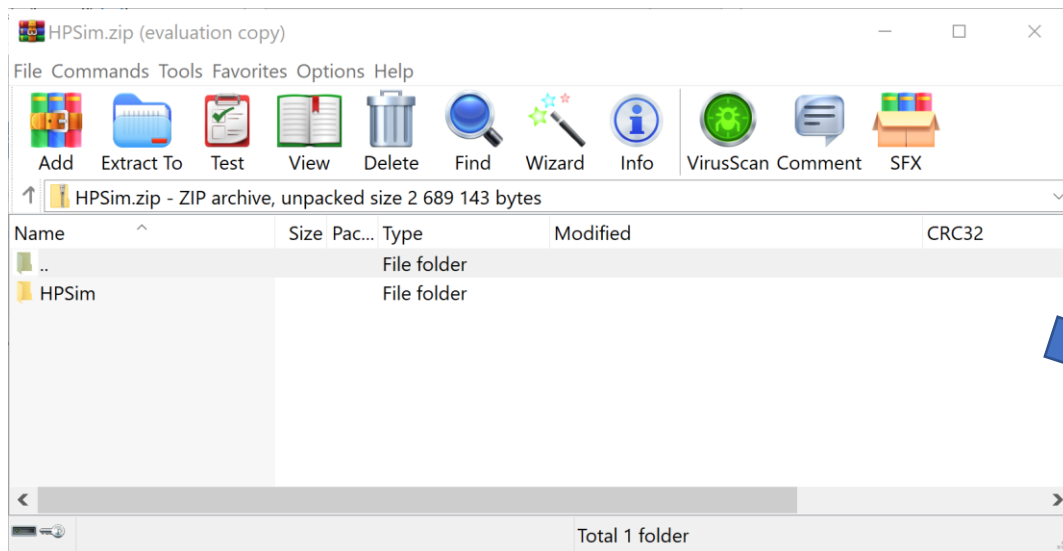
Part 1

Software installation and development of first PN models

- Use theoretical slides for Petri net conceptualization.

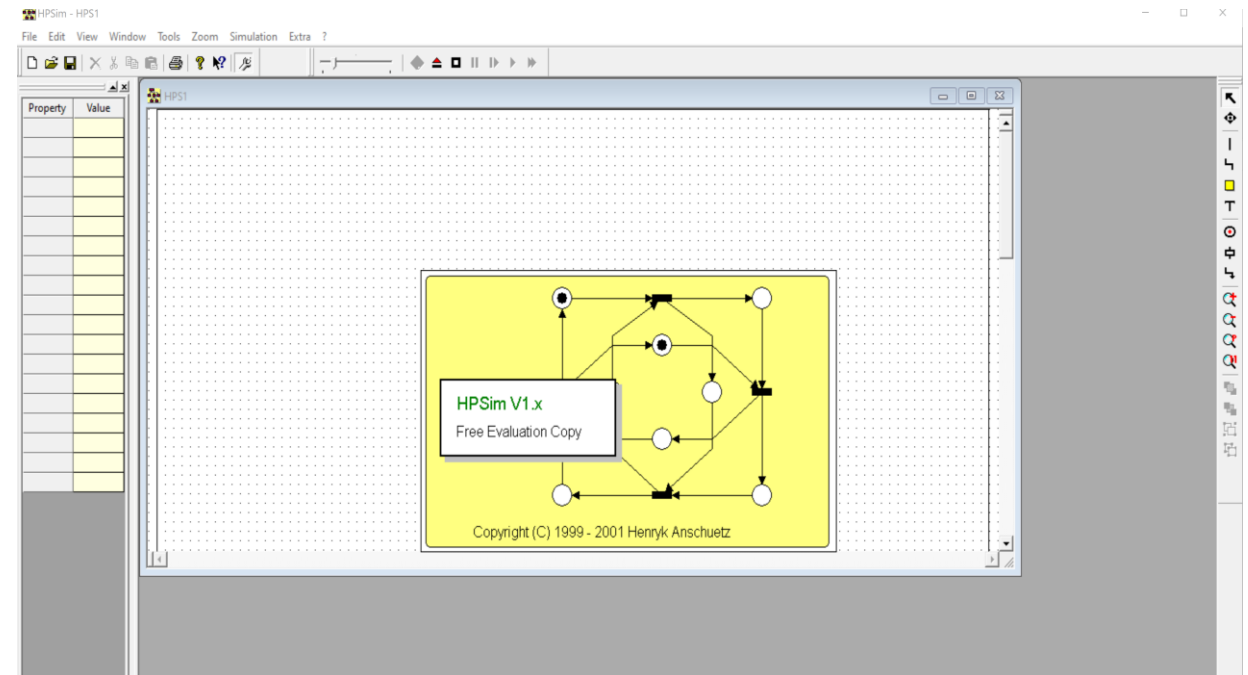
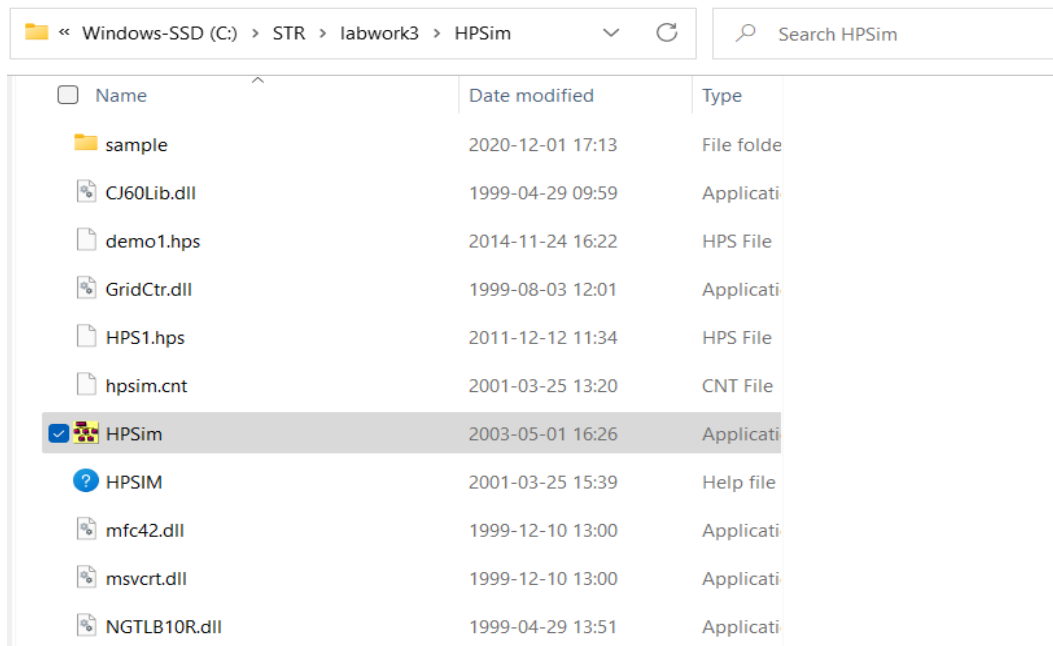


- ❑ Download the package **HPSim.zip** from CLIP
- ❑ Extract it inside **c:\str\labwork3** (strongly recommend it, posterior phases use absolute paths inside this folder)



RUNNING HPSim

- Although a bit old, the software has zero footprint on windows (no libraries spread inside system folders).
- To run it, just click in **HPSim.exe**.

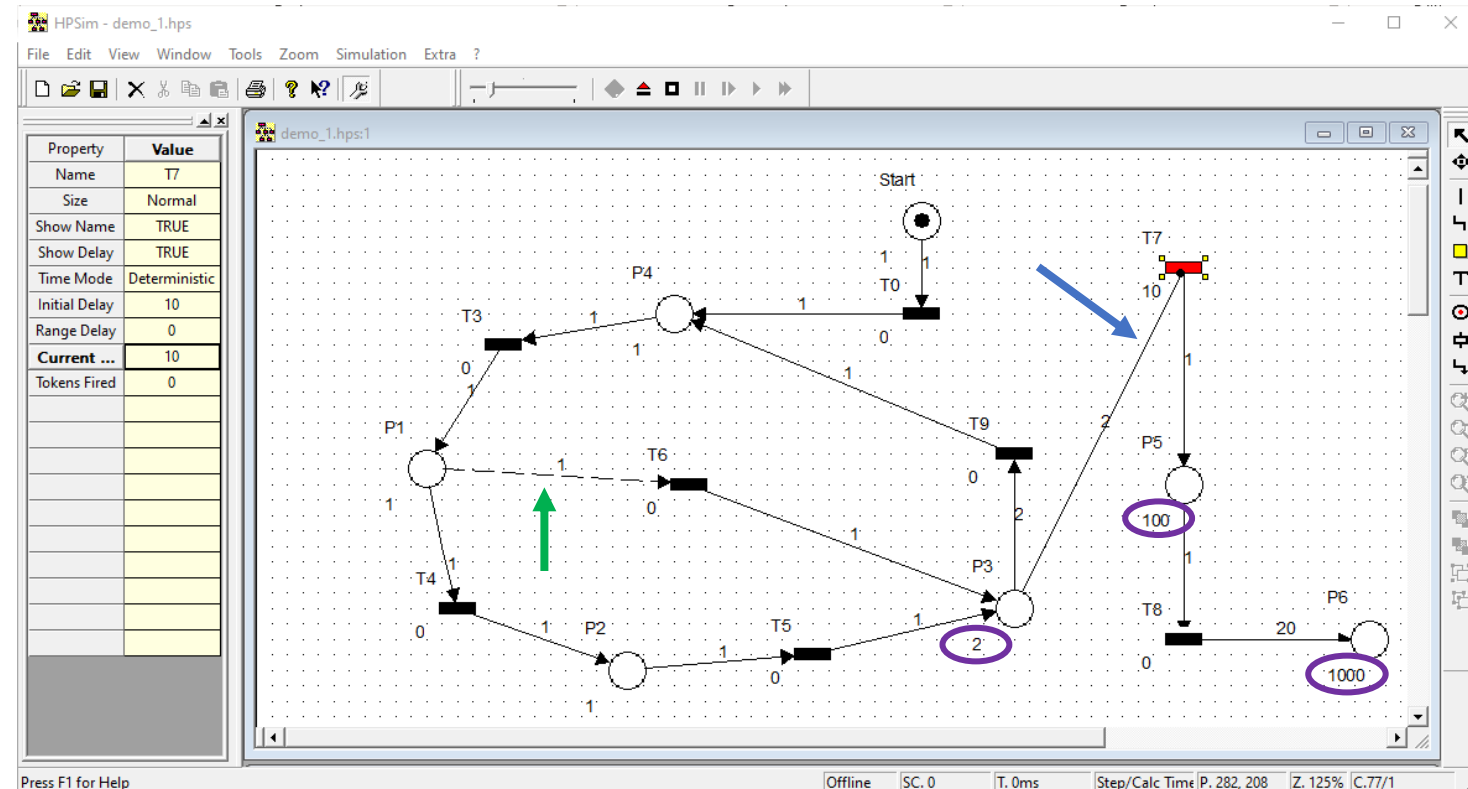


- ❑ To learn the software, we can rely on good external sources.
- ❑ See for instance, the manual available in CLIP and also in http://www.dee.ufrj.br/control_automatiko/cursos/Manual_HPSim.pdf (inside the document, the link for hp download is broken)
- ❑ A video (Spanish language) illustrating the HPSim capabilities is here: <https://www.youtube.com/watch?v=DG2iEkuqcaU>

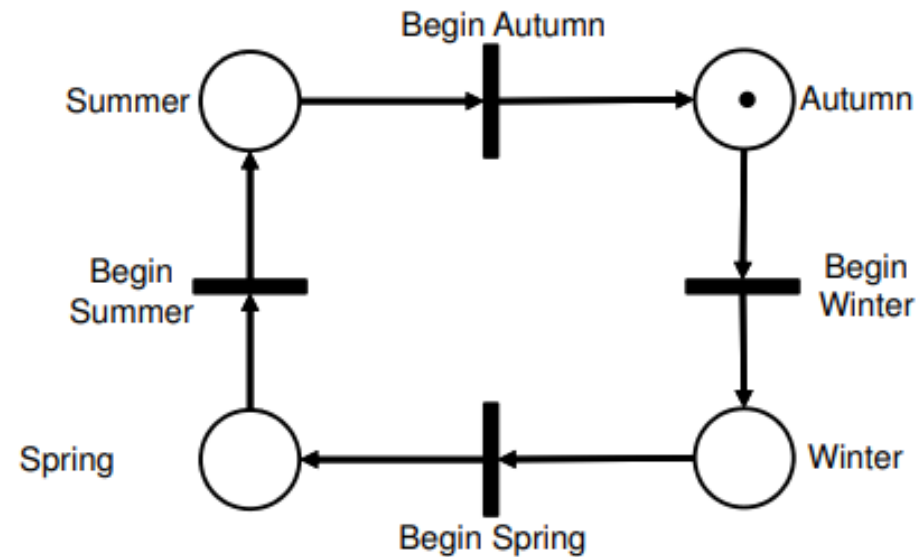
- ❑ It is not frequent, but sometimes the software crashes.
- ❑ Therefore, while creating the Petri Net diagrams, save the work regularly.

ILLUSTRATIVE EXAMPLE

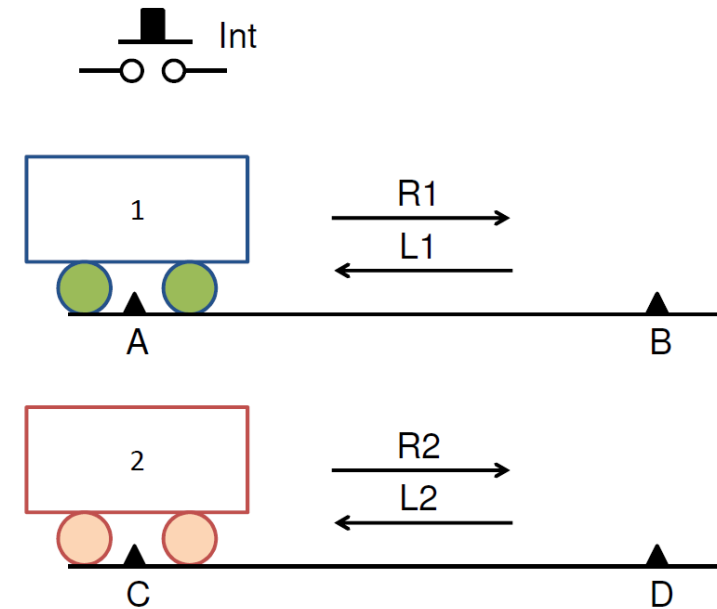
- The example shown in the picture illustrates a model containing:
 - Immediate and **timed** transitions
 - Places with variable **capacity**
 - Normal, **Testing** and **inhibitor** arcs
- Develop this model in your laptop and simulate to observe its behavior.



- Model and observe the behavior of the petri-net shown in the picture.



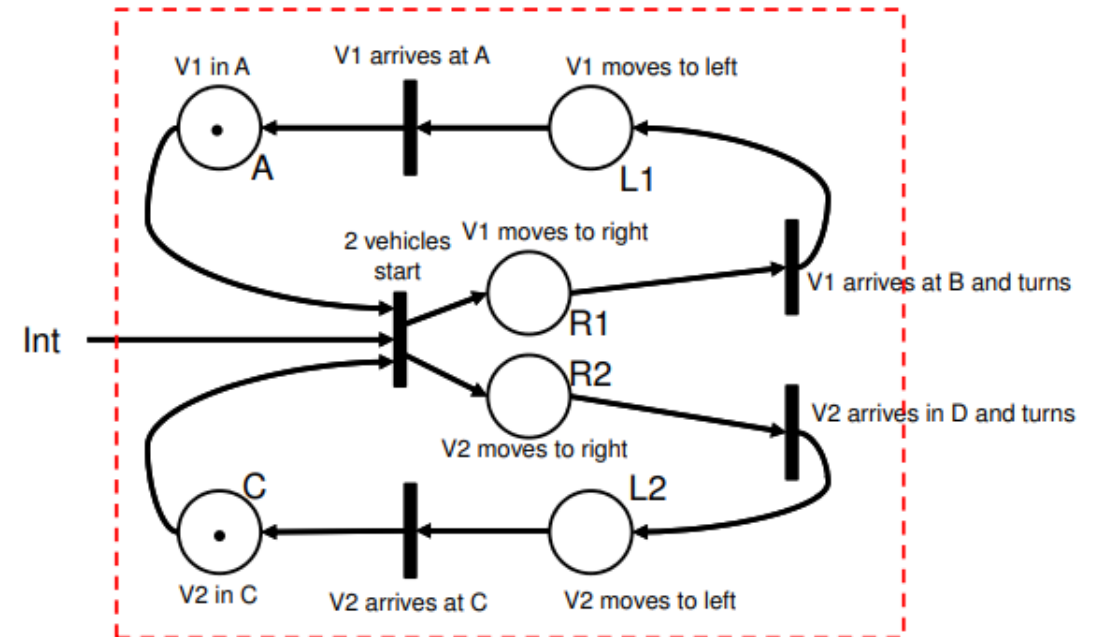
- ❑ Two vehicles can move along two paths with different speeds.
- ❑ Closing **Int**, both vehicles start moving to the right.
- ❑ When a vehicle arrives at the end (B or D), it starts moving to the left, stopping in the initial position (A or C).
- ❑ The same cycle can be repeated whenever both vehicles are in the initial position and the switch **Int** is closed.



Represent this case in PN using HPSim.

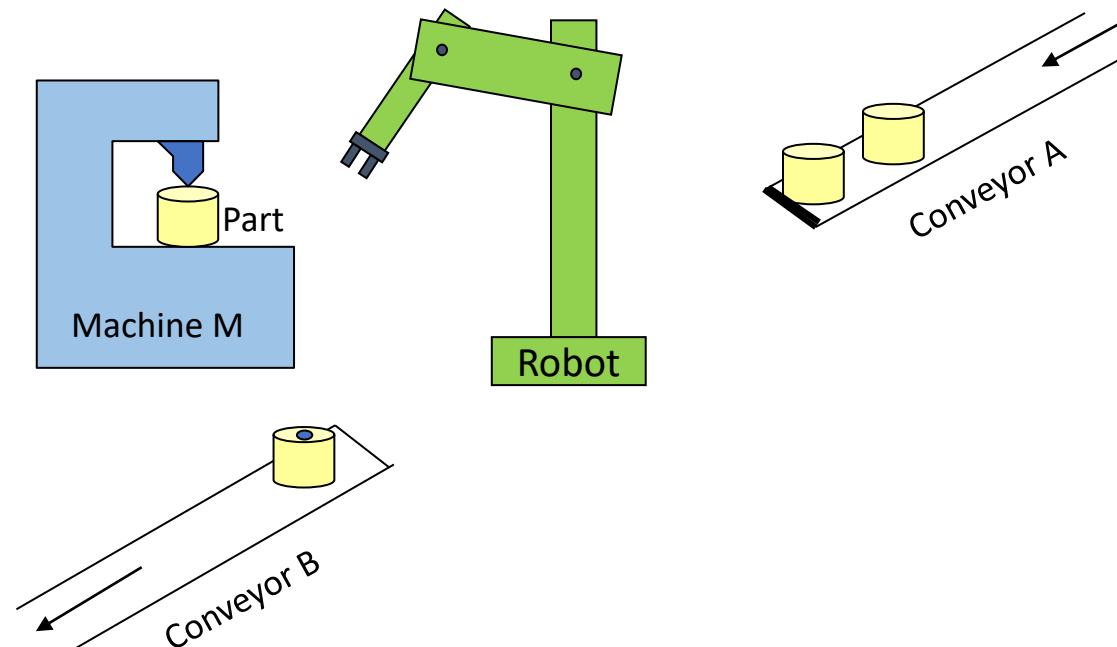
- ❑ Use a place with a token for modeling the “int” requests
- ❑ Try to conceive a solution for modeling more frequent requests

One solution:



L.M. Camarinha-Matos 2019

- ❑ Consider the following system. Parts to be processed arrive in conveyor A. The robot picks one part at a time from that conveyor and puts it in machine M for processing (when both the robot and the machine are free). When the machine finishes, the robot picks the processed part and puts it in the exit conveyor B.
- ❑ Describe the operation of this system using Petri nets. Define the initial marking corresponding to the situation shown in the figure.



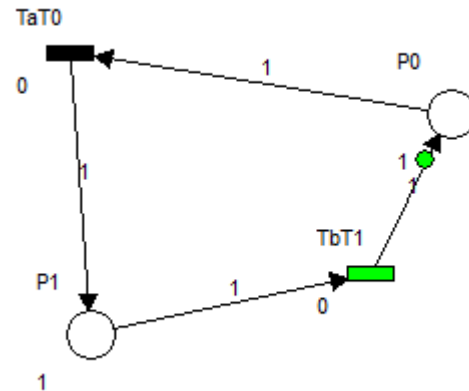
- ❑ We are now going to develop several models which allow training on the Petri Net concepts.
- ❑ Use the tables in the first section of the **report template (available on clip)** to provide answers.
- ❑ This report with your answers and the other requested information needs to be delivered with the implementation of labwork3.
- ❑ Use HPSim to develop the models for the following cases:
 1. An example of a safe PN, and an example of a bounded PN;
 2. An example of a conservative PN;
 3. A PN with capacity places and weighted arcs;
 4. A PN with capacity arcs of type “testing” and “inhibiting”.
 5. A PN with capacity arcs of type “testing” and “inhibiting”, places capacity and weighted arcs.
 6. A PN which starts working and enters in a deadlock state after a few iterations.

Feel free to use the examples of the theoretical classes and other sources

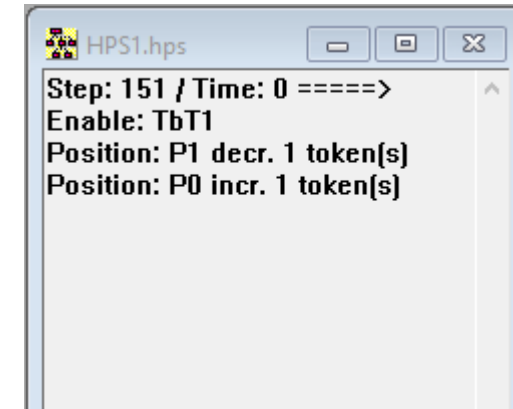


EXAMPLE: MODELING A SECURE PN (DON'T USE THIS AS ANSWER)

- Copy/past the diagram from HPSim in this side and provide brief description of the used scenario/problem.



- Provide some explanations on its behavior during simulation in this side; copy/past the log window.



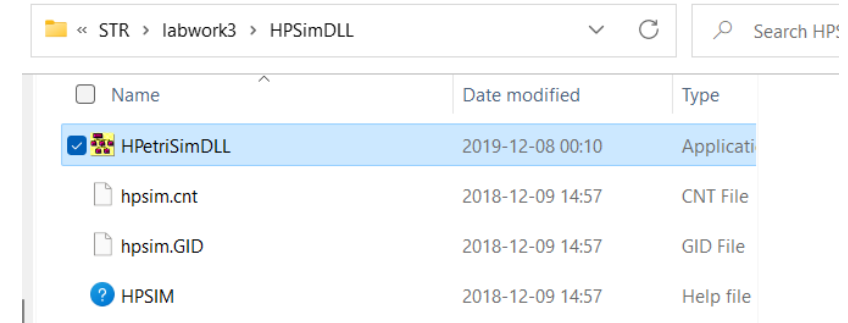
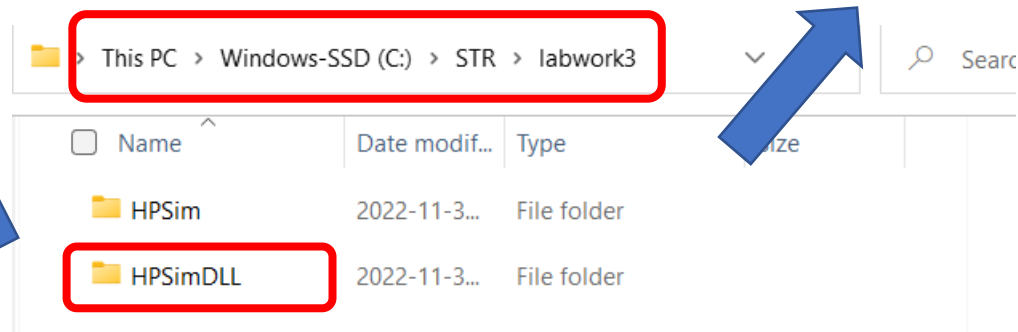
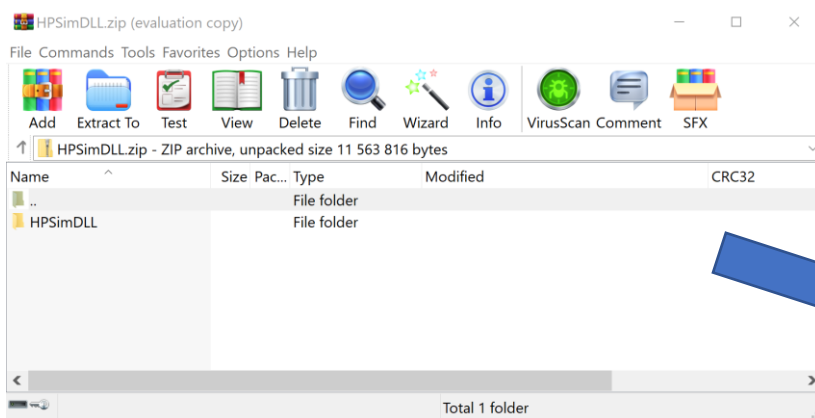
Part 2

DLL integration in HPSim software

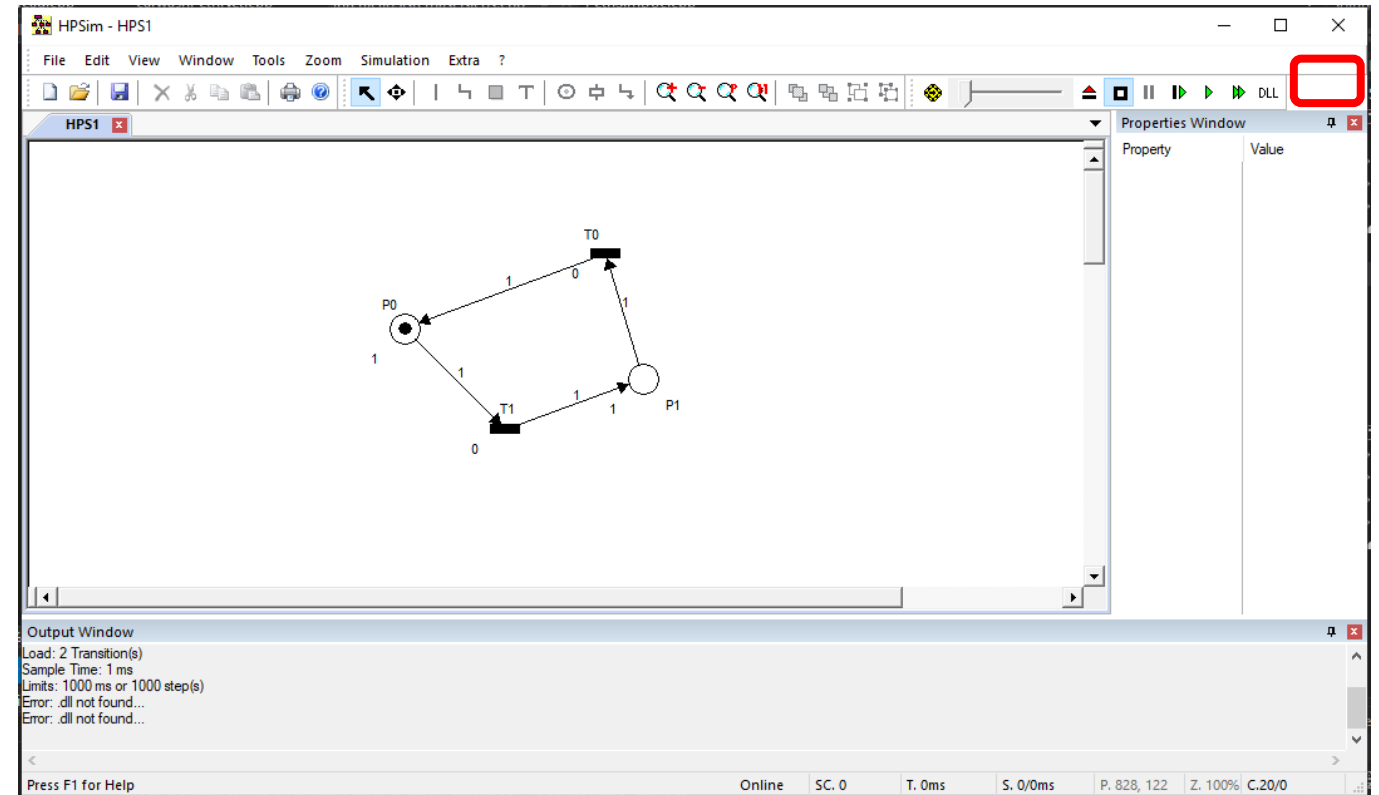
HPSim NEW VERSION INSTALLATION



- ❑ Download the package HPSimDLL.zip from CLIP
- ❑ Extract it inside **c:\str\labwork3** (AGAIN strongly recommended)
- ❑ Launch the executable -> **HPetriSimDLL.exe**

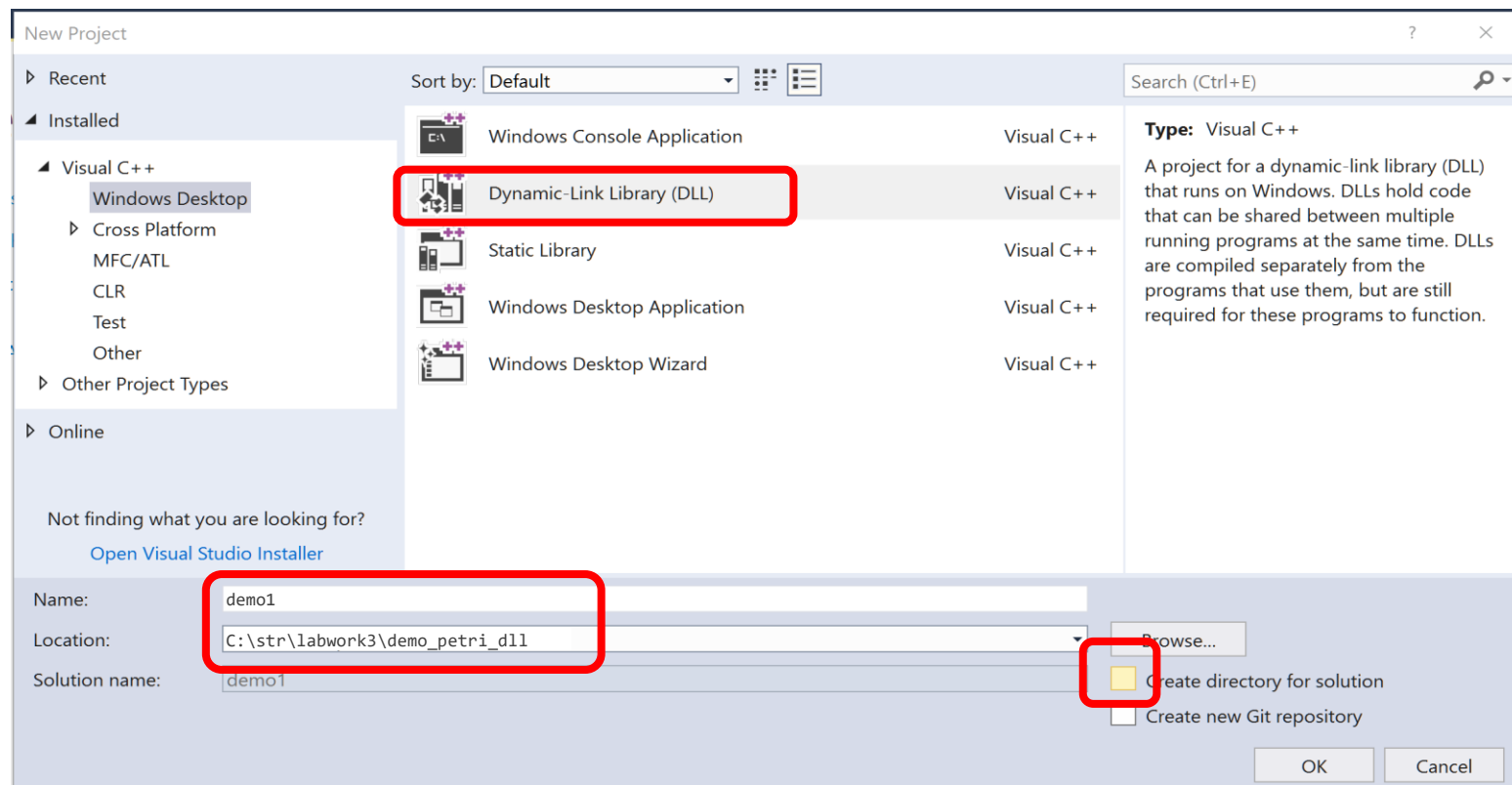


- ☐ This version is pretty much the same.
- ☐ It has a new functionality which allows to associate a DLL to a Petri Net diagram.
- ☐ This way, HPSim can switch between PN simulator and PN executor.



CREATING DLL (VS2017)

- ❑ Launch Visual Studio
- ❑ Select: Create New Project -> Windows Desktop -> Dynamic-Link Library



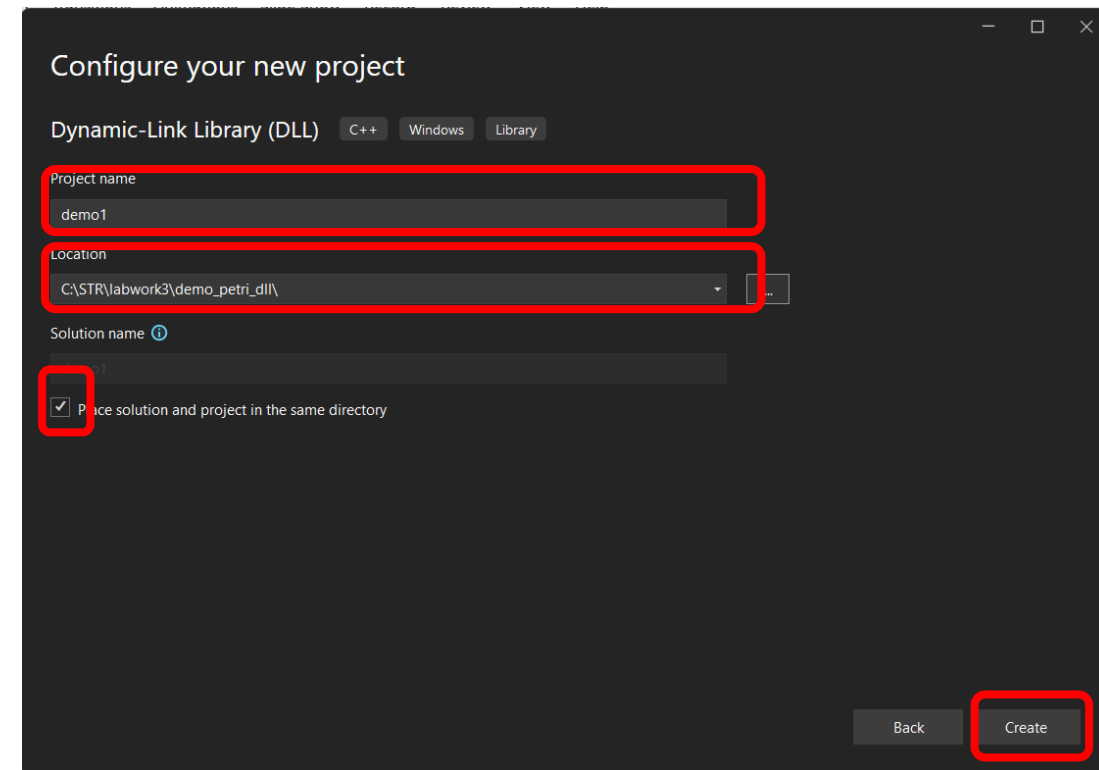
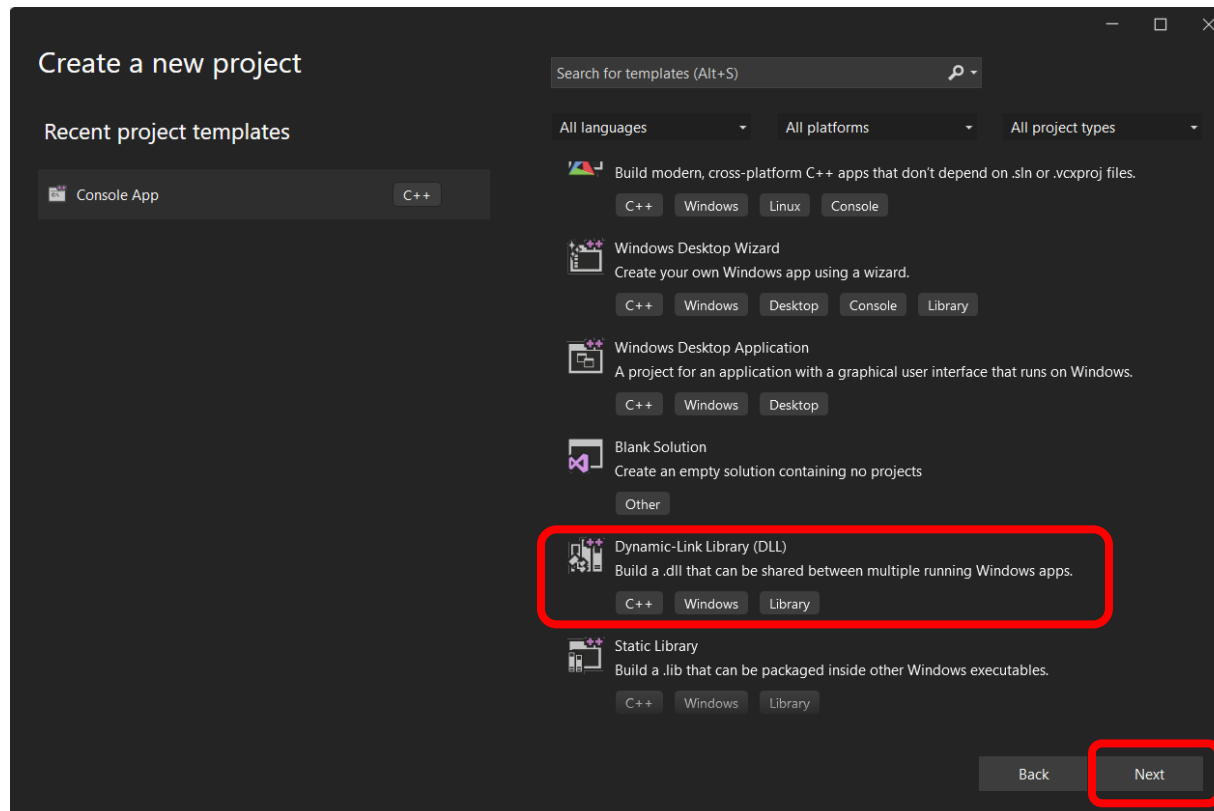
- Project Name: demo1

- Location: c:\str\labwork3\demo_petri_dll

CREATING DLL (VS2019 / VS2022)



- ❑ Launch Visual Studio
- ❑ Select: Create a New Project -> Dynamic-Link Library (DLL)



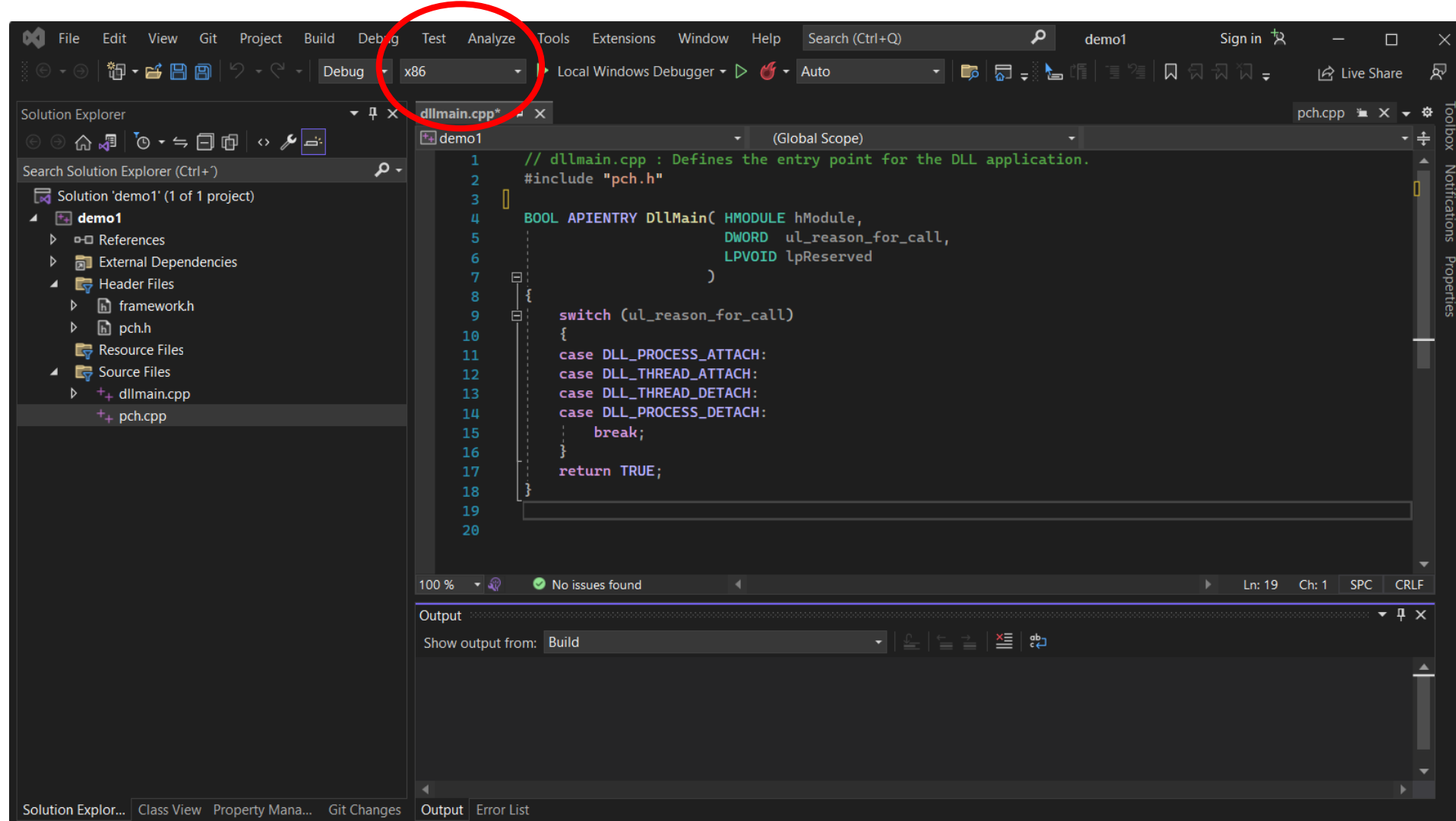
- Project Name: demo1

- Location: c:\str\labwork3\demo_petri_dll

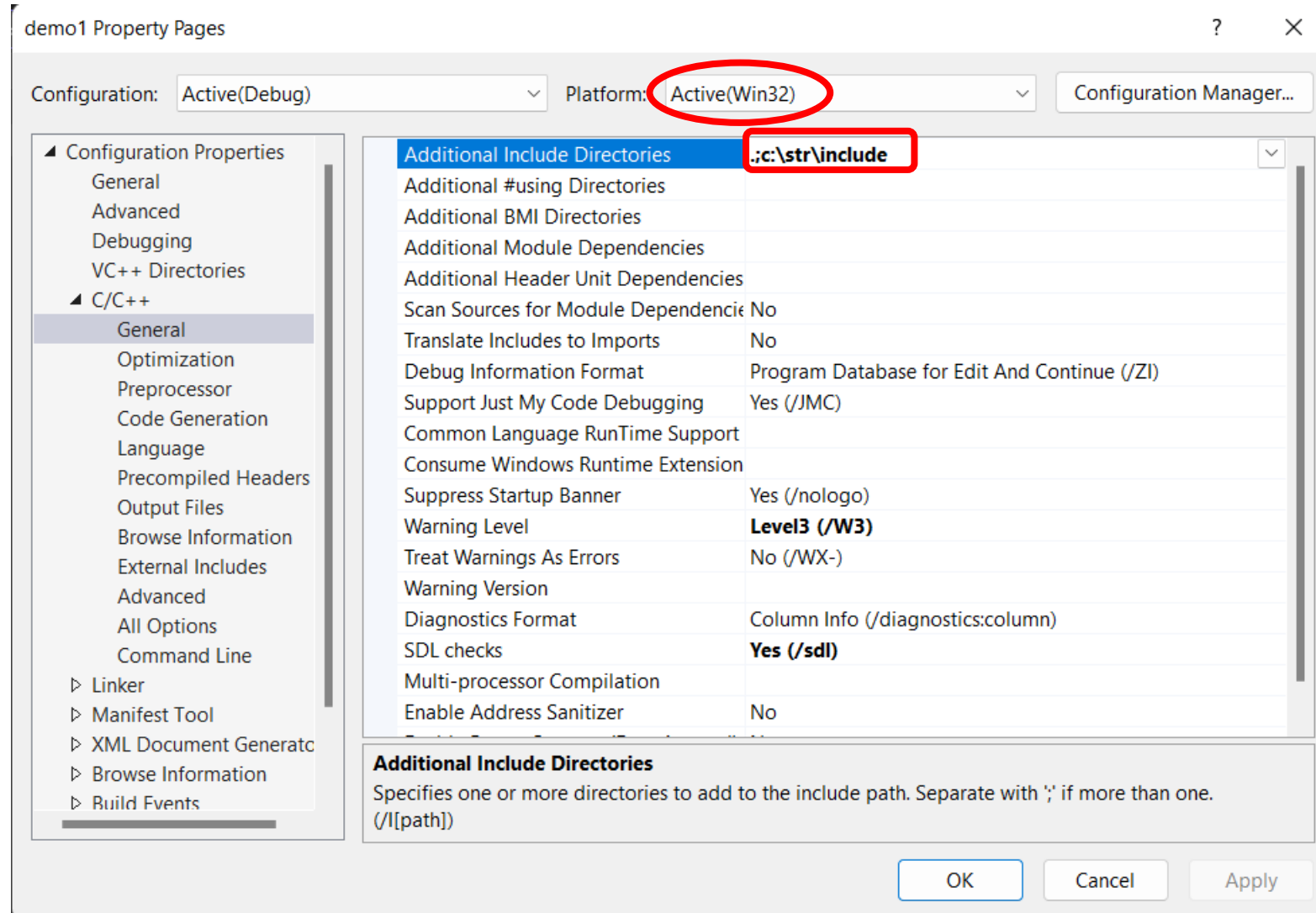
ASPECT OF CREATED DLL PROJECT



x86!!



CONFIGURATION – “includes”

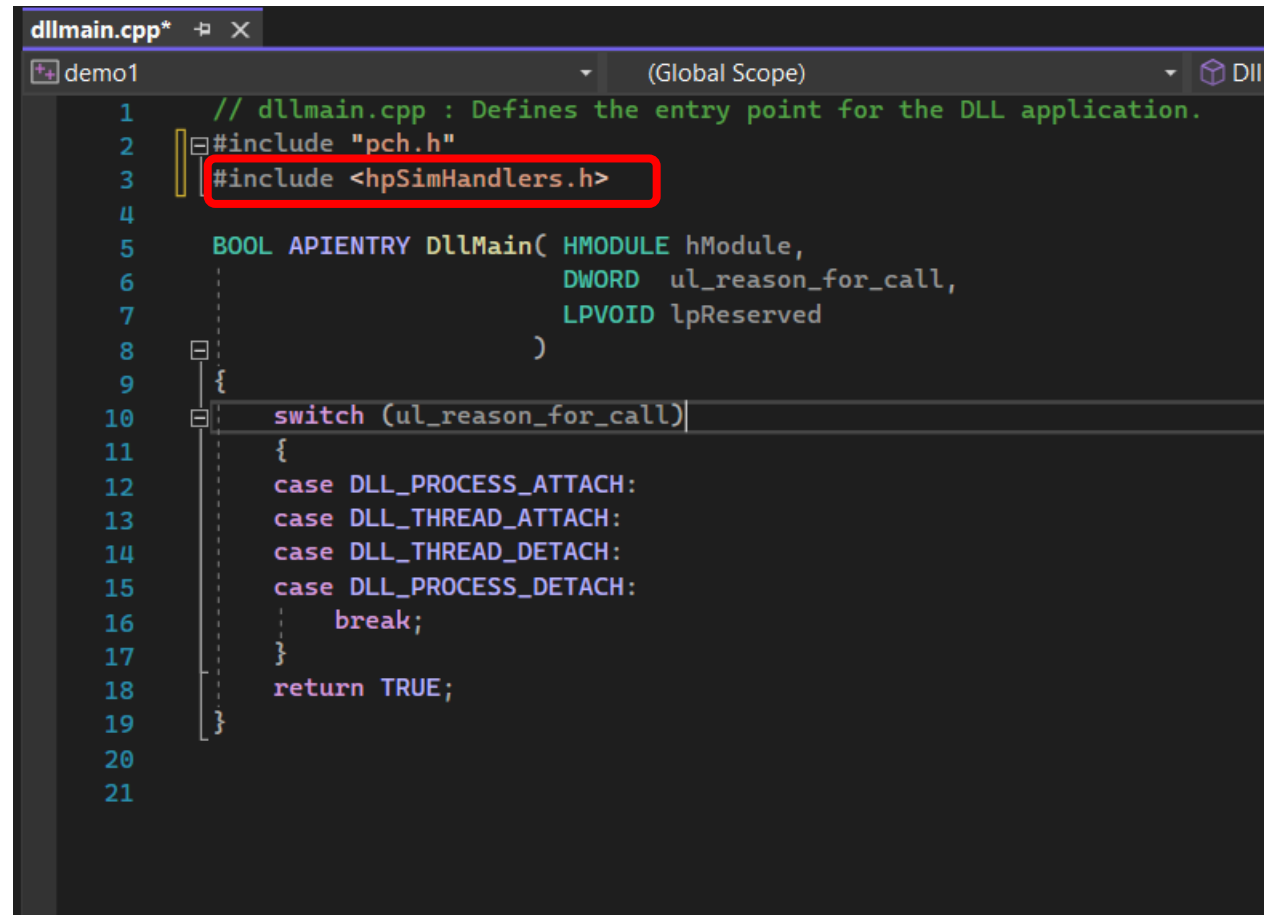


.;c:\str\include

CONFIGURATION – “includes”

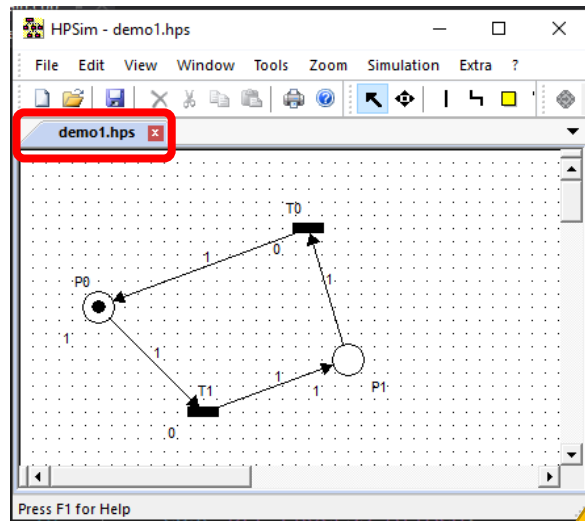


- ❑ Download the file **hpSimHandlers.h** from CLIP and save it into **c:\str\include**.
- ❑ Include it in **dllmain** source file and **build** the solution (not run!)

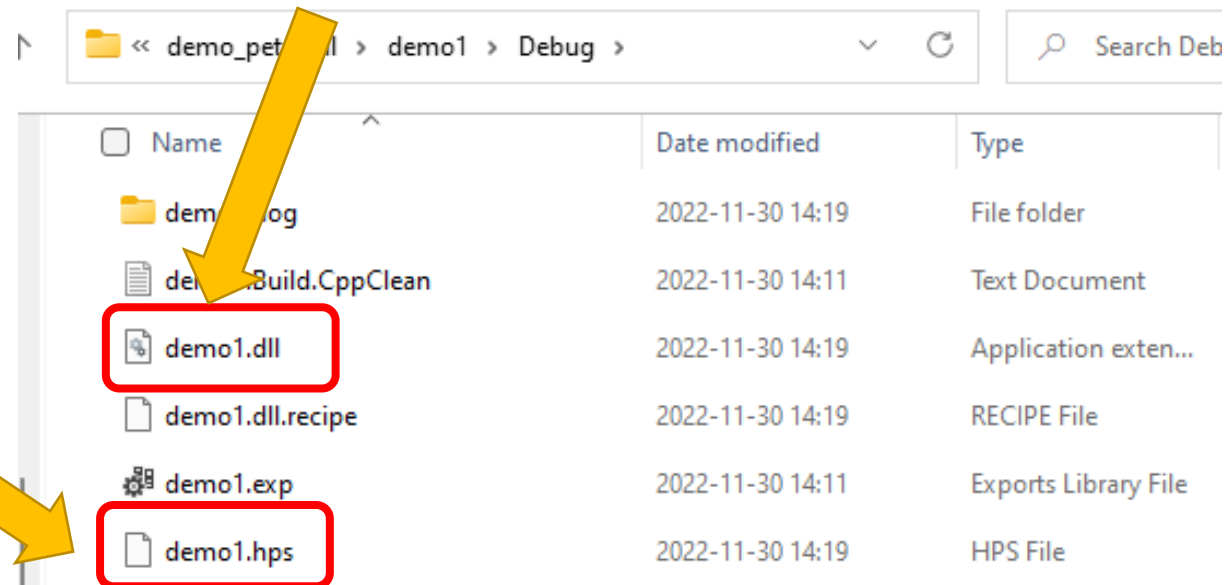
A screenshot of a code editor window titled 'dllmain.cpp*'. The editor shows the source code for a DLL's entry point. Line 3, which contains the line '#include <hpSimHandlers.h>', is highlighted with a red rectangular box. The code includes a comment on line 1, a precompiled header on line 2, and a switch statement starting on line 10. The switch statement handles different reasons for being called, with cases for DLL_PROCESS_ATTACH, DLL_THREAD_ATTACH, DLL_THREAD_DETACH, and DLL_PROCESS_DETACH, followed by a break statement and a return TRUE statement.

```
1 // dllmain.cpp : Defines the entry point for the DLL application.
2 #include "pch.h"
3 #include <hpSimHandlers.h>
4
5 BOOL APIENTRY DllMain( HMODULE hModule,
6                       DWORD ul_reason_for_call,
7                       LPVOID lpReserved
8                       )
9 {
10     switch (ul_reason_for_call)
11     {
12     case DLL_PROCESS_ATTACH:
13     case DLL_THREAD_ATTACH:
14     case DLL_THREAD_DETACH:
15     case DLL_PROCESS_DETACH:
16         break;
17     }
18     return TRUE;
19 }
20
21
```

- Look at the output of the DLL in VStudio.



```
Output
Show output from: Build
Build started...
1>----- Build started: Project: demo1. Configuration: Debug Win32 -----
1>demo1.vcxproj -> C:\STR\labwork3\demo_petri_dll\demo1\Debug\demo1.dll
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
```



Name	Date modified	Type
demo1.log	2022-11-30 14:19	File folder
demo1.Build.CppClean	2022-11-30 14:11	Text Document
demo1.dll	2022-11-30 14:19	Application exten...
demo1.dll.recipe	2022-11-30 14:19	RECIPE File
demo1.exp	2022-11-30 14:11	Exports Library File
demo1.hps	2022-11-30 14:19	HPS File

- Save the Petri Net file in the **same folder and same name** as the compiled DLL (**IMPORTANT**).

INTEGRATION WITH PETRI NET (2)



- ❑ Copy – paste the code below inside the dllmain.cpp (before `BOOL APIENTRY DllMain`)

```
#include "pch.h"
#include <hpSimHandlers.h>

char logHpSim[512] = "";
LOG_HANDLER getLogString(void)
{
    return logHpSim;
}

void sprintfLog(const char* message) {
    strcpy_s(logHpSim, sizeof(logHpSim), message);
}

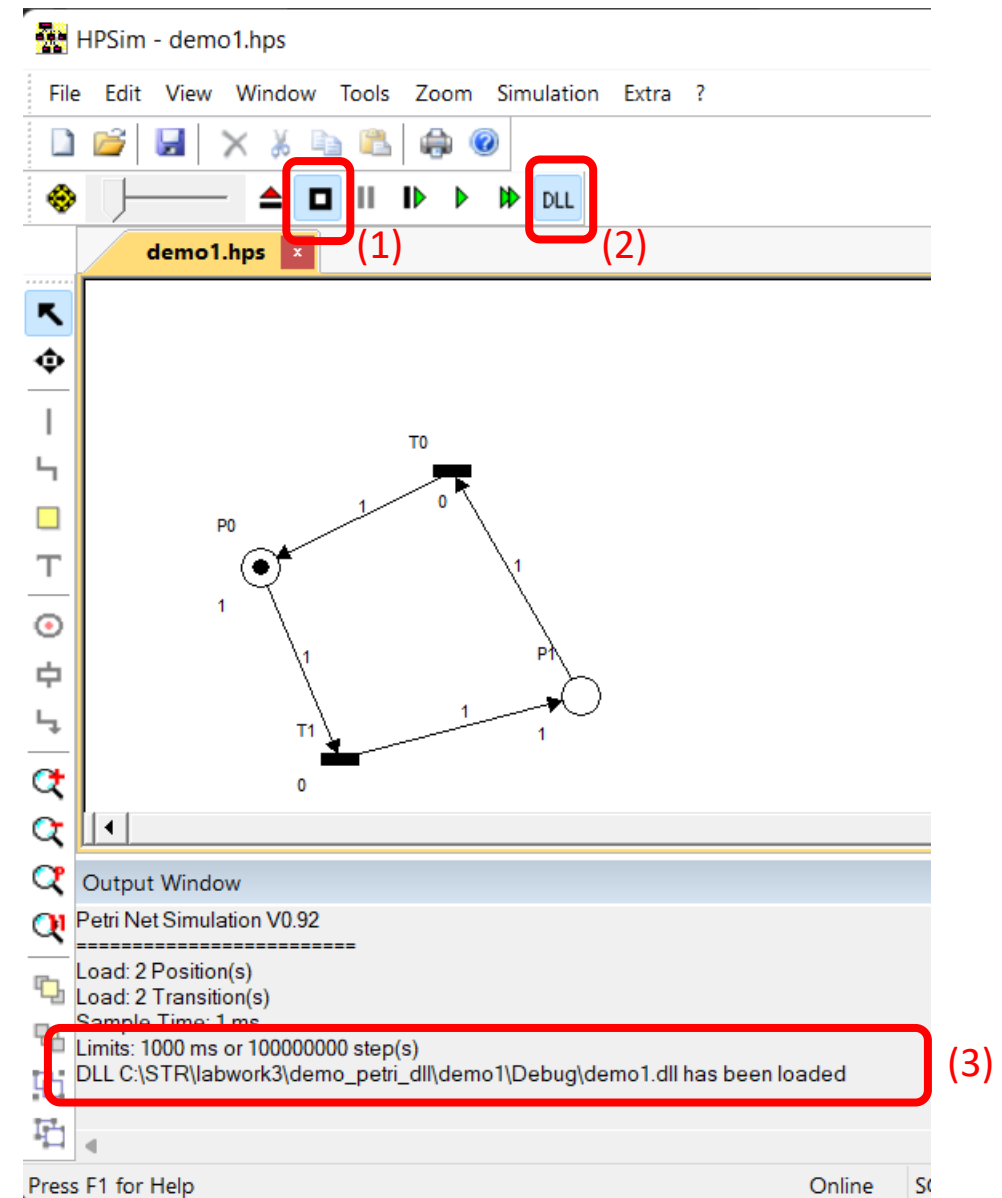
TRANSITION_HANDLER T0() { sprintfLog("It is in transition T0"); return true; }
PLACE_HANDLER      P0() { sprintfLog("It is in Place      P0"); return true; }
```



```
dllmain.cpp*  demo1  (Global Scope)
1  // dllmain.cpp : Defines the entry point for the DLL application.
2  #include "pch.h"
3  #include <hpSimHandlers.h>
4
5  char logHpSim[512] = "";
6  LOG_HANDLER getLogString(void)
7  {
8      return logHpSim;
9  }
10
11 void sprintfLog(const char* message) {
12     strcpy_s(logHpSim, sizeof(logHpSim), message);
13 }
14
15 TRANSITION_HANDLER T0() { sprintfLog("It is in transition T0"); return true; }
16 PLACE_HANDLER      P0() { sprintfLog("It is in Place      P0"); return true; }
17
18
19
20
21 BOOL APIENTRY DllMain( HMODULE hModule,
22                       DWORD ul_reason_for_call,
```

PREPARING SIMULATION

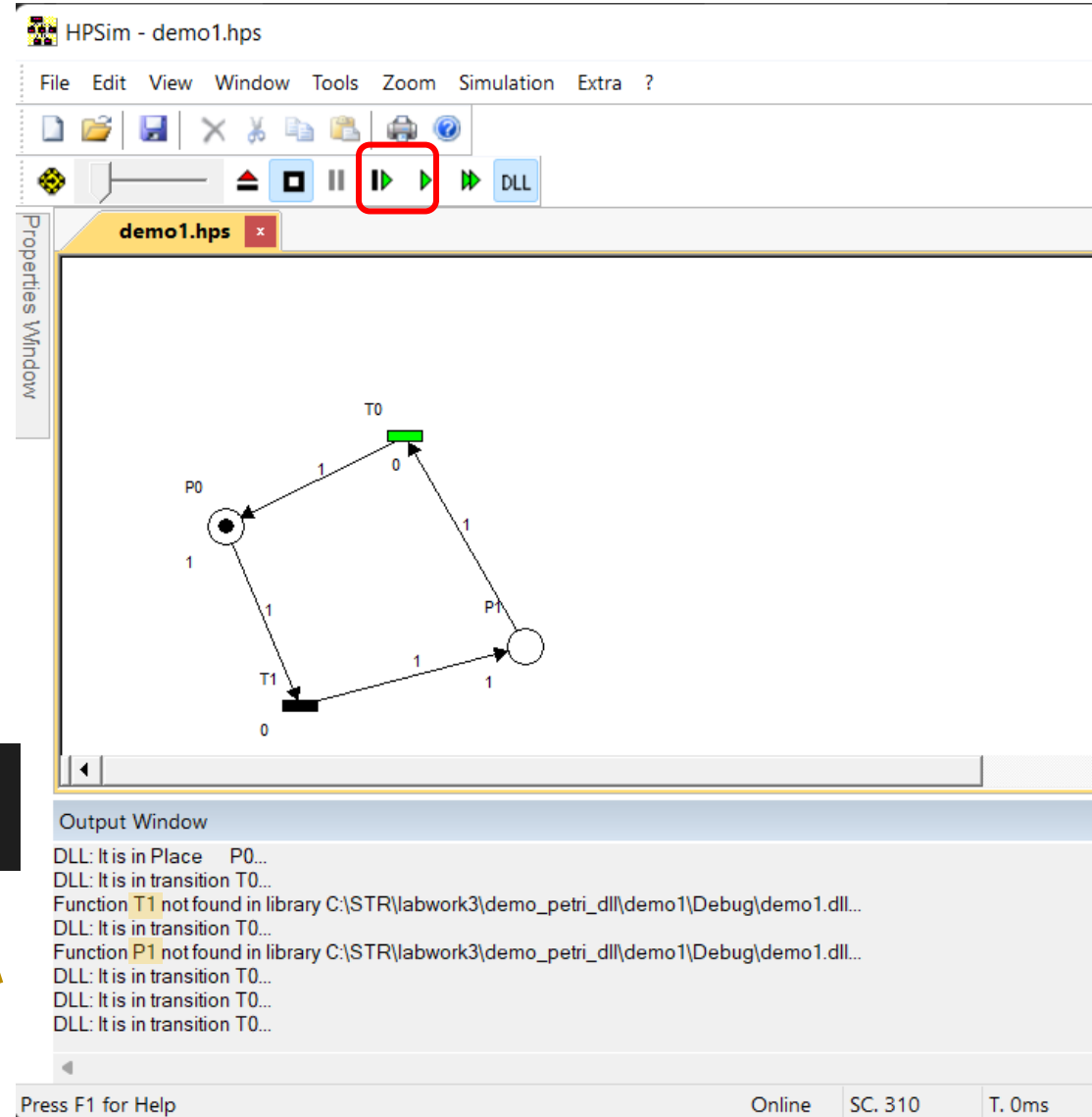
- ❑ Each time the DLL code is changed, it should be compiled; do it now.
- ❑ Now go into HPSim and get into simulation mode (1)
- ❑ Then, connect the model to the DLL by pressing DLL button (2).
- ❑ Notice the message in the Output Window telling “DLL:... dem1.dll has been loaded” (3)
- ❑ If not, ensure that both **demo1.hps** and **demo1.dll** have got the same name and in the same folder.



RUNNING THE PETRI NET

- Now press the play button.
- Notice how the log messages from the handlers appear in the output window.
- Furthermore, transitions and places that were not assigned with a handler in the DLL are identified by corresponding alert message.

```
TRANSITION_HANDLER T0() { sprintfLog("It is in transition T0"); return true; }  
PLACE_HANDLER      P0() { sprintfLog("It is in Place      P0"); return true; }
```



The screenshot shows the HPSim - demo1.hps application window. The menu bar includes File, Edit, View, Window, Tools, Zoom, Simulation, and Extra. The toolbar contains various icons, with the play button (a green right-pointing triangle) highlighted by a red rectangle. Below the toolbar, the main window displays a Petri net diagram with three places (P0, P1) and two transitions (T0, T1). P0 contains one token. Transitions T0 and T1 are highlighted in green. The output window at the bottom shows log messages from the DLL handlers:

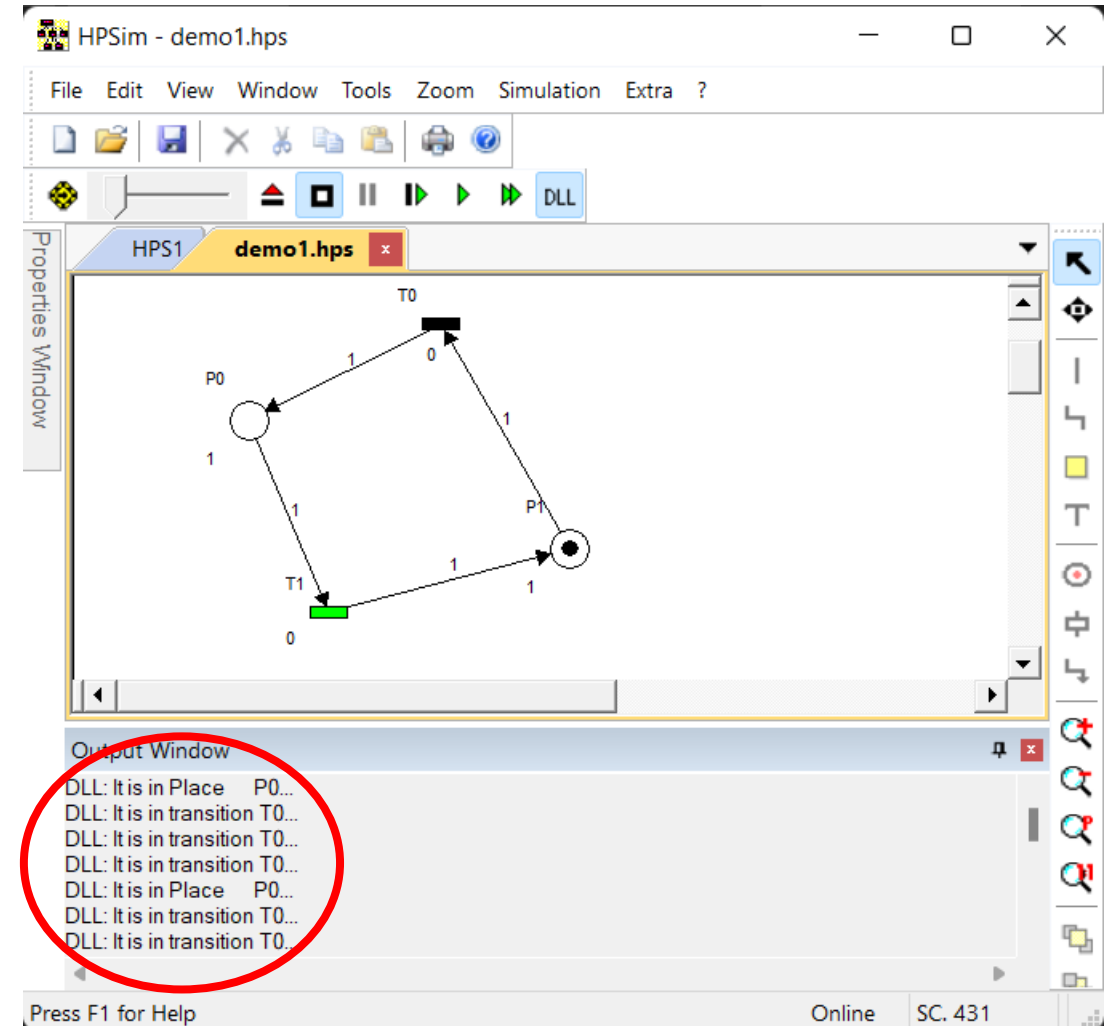
```
DLL: It is in Place P0...  
DLL: It is in transition T0...  
Function T1 not found in library C:\STR\labwork3\demo_petri_dll\demo1\Debug\demo1.dll...  
DLL: It is in transition T0...  
Function P1 not found in library C:\STR\labwork3\demo_petri_dll\demo1\Debug\demo1.dll...  
DLL: It is in transition T0...  
DLL: It is in transition T0...  
DLL: It is in transition T0...
```

At the bottom of the window, there is a status bar with the text "Press F1 for Help", "Online", "SC. 310", and "T. 0ms".

DISABLING MISSING HANDLER MESSAGES

- ❑ In order to avoid such messages, you just need to define the functions shown below.
- ❑ As they return false, for now on, it is not necessary to define all the handlers.

```
LOG_ALERT_HANDLER alertMissingTransitions(void) {  
    // returning false prevents the annoying messages  
    return false;  
}  
  
LOG_ALERT_HANDLER alertMissingPlaces(void) {  
    // returning false prevents the annoying messages  
    return false;  
}
```



- Beyond the usual petri net rules for firing transitions, we can use Boolean conditions, based on e.g., sensors.
- In such a way, handlers of transitions will encode these conditions. A transition would fire if all places connected to it have the necessary tokens and Boolean condition yields true:

```
TRANSITION_HANDLER T0() {  
    sprintfLog("It is in transition T0");  
    uInt8 p = readDigitalU8(1);  
    bool partDetected = (bool)getBitValue(p, 3);  
    return partDetected;  
}
```

- ❑ We can associate actions in C++ to the places in a PN
- ❑ Whenever a place has a token, the corresponding action is executed.
- ❑ When a handler of a place returns **true**, the action is executed just **once**
- ❑ When a handler of a place returns **false**, the action is executed in **every** step of the execution.

```
PLACE_HANDLER P0() {  
    sprintfLog("It is in Place      P0");  
    stopVerticalMovement();  
    moveBrushRight();  
    return true; // execute only once  
}
```

- ❑ Other handlers can be associated to Places; they are:

```
TOKEN_CHANGED_HANDLER onTokenEnterPlace_P0(long tokensBefore, long tokensNow) {  
    if (tokensBefore == 0 && tokensNow > 0) {  
        //do something here  
    }  
}  
  
TOKEN_CHANGED_HANDLER onTokenLeavePlace_moveDown(long tokensBefore, long tokensNow) {  
    if (tokensBefore >= 0 && tokensNow == 0) {  
        //do something here  
    }  
}
```

- ❑ These handlers are executed only once, just when a token enters or leaves a Place.

- ☐ Applying the Petri Net modeling in the solution of a Car Wash Services system
- ☐ Installation of CarWash simulator
- ☐ Project interaction with HPSim
- ☐ User interface



KEEP UP THE
GOOD
WORK!